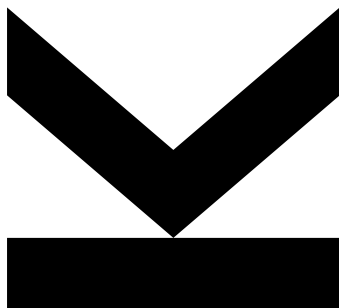


Simon Grundner
Institute for Signal
Processing

@ k12136610@students.jku.at
🌐 <https://jku.at/isp>

October 3, 2025

02 - Finite-State-Machine



Hardwaredesign PR - Exercise 02

Semester 2024W

Keywords Sequential Circuits, FSM

Contents

1. Sequential Circuit	3
1.1 Module Implementation	4
1.2 Simulation	5

1. Sequential Circuit

Task: Sequential circuit

In this exercise a delay circuit should be programmed. After pressing the start button a counter is started and increments until it reaches its maximum value. If this value is reached the output led is activated and the counter is restarted from zero. If the counter reaches its maximum value again, the output led is deactivated and the system waits until the start button is pressed again. Figure 1 shows the blocks which should be implemented to realize the delay circuit. The circuit contains a counter which is used to produce a delay

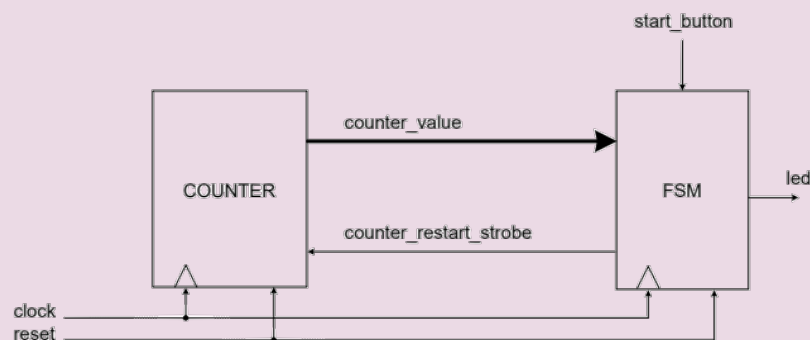


Figure 1: Block diagram of the delay circuit

and a finite-state-machine (FSM) which controls the counter and the interaction with the start button and the led output. Furthermore the needed signals between these two blocks can be seen in figure 1. The signal counter value holds the current counter value, counter restart strobe is used to reset the counter value to zero.

Figure 2 shows the states and transitions of the used finite state machine. The edges are labeled with the events that must happen for a transition into the next state.

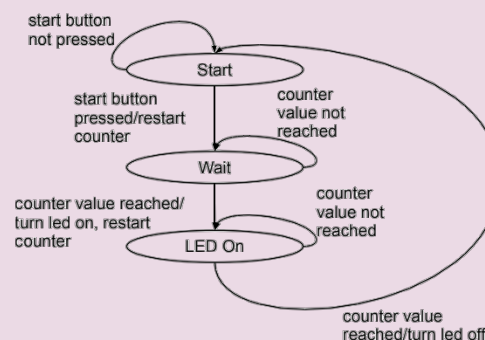


Figure 2: Finite State Machine

To-Do-List

- Write the entities and architectures of the counter and FSM in VHDL. Use one .vhd file for each entity and one .vhd file for each architecture.
- Write a testbench in VHDL (extra file). Simulate the system using Model- sim and hand in screenshots showing the output of the waveform window.
- Discuss the results of the simulation in a few sentences.
- Is the used finite-state-machine a Mealy or Moore machine?

1.1 Module Implementation

The code can be found in the attachments.

Entity

The ports of the modules are taken from the block diagram in figure 1. The in- and output can be determined from the arrow directions.

Architectures*Counter-Architecture:*

The counter architecture consists of a signal, which is used to increment the actual counter output. A register process handles the reset and clock behaviour: On reset, the counter output is set to 0. On each rising edge of the clock signal, the synchronous counter reset strobe is checked. If the strobe is set, the counter is set to 0, else the counter is incremented by one.

Statemachine-Architecture

The functionality of the statemachine is described by two processes: One register process describes the clock and reset behaviour. Here, the state variable is updated every clock cycle. Another combinatorial process determines the next state. Here the counter period can be set by editing the MAX_COUNT constant.

Test-Bench

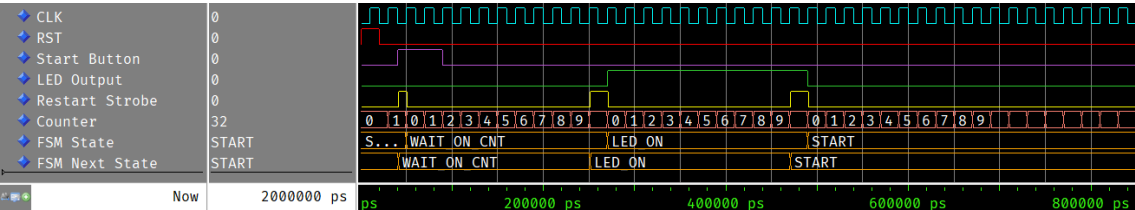
Before the stimulus process a clock oscillation is emulated and a reset is performed. To test the module implementations, two scenarios are tested:

- Pressing the start button at different timestamps
- Performing a synchronous reset

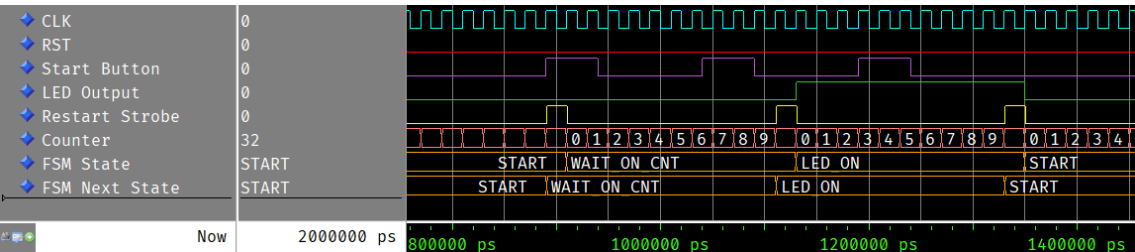
1.2 Simulation

Start Button Stimulus

A single start pulse triggers the statemachine:

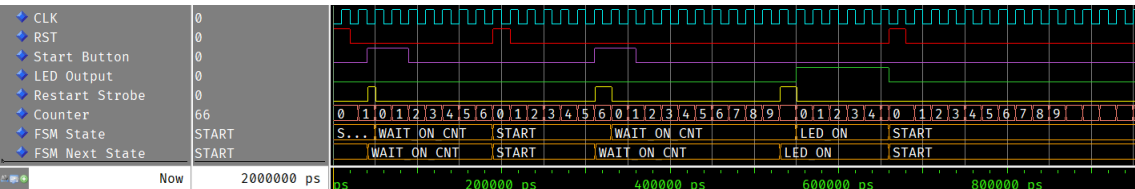


Any further start button press have no effect during the WAIT and the LED_ON state.



Reset Stimulus

The waveform shows, that a synchronous reset during the WAIT and LED_ON cancels the blinking process of the led.



Results

Because the outputs are purely dependent on the states themselves and not on any other inputs, the fsm is per definition a mealy machine.